- Creates a function named `start`. That function, when called, tells the mutation observer to start observing mutations.
- Calls the start function

You are probably wondering why we need to go through the extra step of burying the `observe` call inside a nested function. What we are trying to prevent is a case where a new item is added, it triggers a re-sort, which triggers a mutation, which triggers a re-sort, which triggers a mutation, and on and on. To prevent that infinite loop, we need to shut down observation before we sort things, then restart it after the sorting is done. That's done most easily if the startup is extracted into a function that can be called both when the controller is initialized and then again after the sort is done.

Let's look at the individual pieces of that method in more detail. The constructor for `MutationObserver` takes one argument, a callback function that is invoked when the observer is both active and detects a change.

By itself, creating the `MutationObserver` object does nothing. You need to also tell it to observe something; we do in the `start` method with `observer.observe(target, { childList: true, subtree: true })`. The `observe` method takes two arguments. The first is the target element to be observed, in this case `target`, which is the element that the controller is declared as part of. The observer looks for changes within that target element. The second argument is an options object, which is a collection of true/false flags that tell you what kinds of mutations should trigger the callback.

The callback function is invoked when a change happens that matches the kind of changes that `observe` is watching for. The purpose of the callback is to call the `sortTargets` method of the controller. The arguments to the callback are a list of mutation records that store information about each change and the newly created mutation observer itself, though in our case, we don't actually need information about the changes.

Around the call to `sortTargets`, we are disconnecting and then reconnecting the observer to prevent an infinite loop of changes triggering changes. Well…